Parcial práctico de Ingeniería de Software

La última transmisión: motor de historias interactivas

Universidad Tecnológica de Bolívar · 14 de octubre de 2026

Integrantes	Repositorio	Versiones
Sebastián Felipe Caicedo Acosta Javier Alejandro Carta Lacharme	github.com/sebastian-caicedo/parcial_ultima_transmision	v1.0-pre-cambio (hora 6) v2.0-post-cambio (entrega)

Datos de referencia: el archivo oficial la_ultima_transmision.json se publicó en el aula después de la hora 6. Hasta entonces se trabajó con una transcripción propia de las tablas de la guía; al recibirlo se reemplazó por el oficial (solo cambiaron dos IDs de opción: revisar y activar_codigo) y se ajustaron las pruebas. El motor no requirió cambios.

Los diagramas, la matriz de trazabilidad completa y las capturas del tablero están en el **Anexo** (Anexo_Parcial.pdf). El detalle de cada sección está en docs/ del repositorio.

1. Problema, actores, alcance y calidad

Un colectivo de escritores prepara una ficción interactiva para un festival universitario. Los escritores no saben programar pero quieren agregar escenas y cambiar decisiones sin pedir cambios al programa. El problema es construir un **motor que interprete historias guardadas en archivos JSON**: que muestre solo las opciones permitidas por lo que ha pasado, aplique sus consecuencias y avise con claridad, antes de jugar, cuando un archivo tiene errores.

Actor	Tipo	Necesidad
Escritor	Usuario	Editar historias sin tocar código y saber qué corregir
Lector	Usuario	Ver solo opciones válidas, elegir, reconocer el final, reiniciar y deshacer
Diseñador de la experiencia	Interesado	Decisiones con consecuencias y opciones condicionadas
Coordinadora del festival	Interesada	Que el programa no se cierre y los errores se detecten antes
Sistema de archivos	Externo	Fuente de las historias (solo lectura)

Alcance. Incluye: cargar y validar historias, opciones condicionadas, efectos booleanos, finales, bloqueo, reinicio y, desde la hora 6, deshacer un nivel. No incluye: cuentas, base de datos, guardado entre ejecuciones, internet, editor visual, multimedia ni lenguaje de expresiones.

Riesgo	Mitigación
R1. Los escritores editan el JSON a mano y pueden introducir errores que hagan fallar el programa en el festival	Validación completa antes de abrir la partida (V01–V09), mensajes con código e ID, una prueba por categoría
R2. Cambio de requisitos en la hora 6 que obligue a rehacer código	Capas separadas para localizar el cambio; versión etiquetada v1.0-pre-cambio y análisis de impacto

Priorización de calidad: 1) **confiabilidad** (la coordinadora no acepta cierres inesperados y una opción no habilitada nunca debe ejecutarse, CA02); 2) **modificabilidad del contenido** (razón de ser del producto); 3) **claridad de errores** (el escritor no programa); 4) mantenibilidad, interna al equipo; 5) rendimiento, poco relevante con historias de decenas de escenas. Los tres primeros originan los RNF.

Decisión de responsabilidad profesional (autoría). El motor trata las historias como **solo lectura**: nunca las corrige ni completa campos. Si arreglara en silencio un destino roto, se publicaría en el festival una historia que nadie escribió. El software informa y el autor decide (verificado en test_validar_no_modifica_los_datos). Además se usa solo la biblioteca estándar de Python; pytest (MIT) solo para pruebas.

2. Proceso y agilidad

Proceso elegido: incremental con Kanban. Cada incremento deja algo ejecutable y probado: validador, motor, consola, cambio. Frente a **cascada**, que deja el código para el final y obliga a

reabrir fases cerradas cuando llega el cambio de la hora 6, el incremental siempre tiene una versión entregable. Scrum se descartó: sprints y ceremonias no tienen sentido en 12 horas con dos personas.

Actividad	Cómo se realizó	Evidencia
Especificación	RF/RNF desde N01–N06 antes del primer incremento; dudas como supuestos	docs/02_requerimientos.md
Desarrollo	Un incremento por vez, tarea en el tablero, commits pequeños con ID	Historial de Git
Validación	pytest en cada incremento; CA01–CA03; recorrido de finales	tests/, evidencias/pruebas/
Evolución	Tag en la hora 6, análisis de impacto, cambio como nuevo incremento	docs/05_solicitud_cambio.md

Backlog y tablero: 37 tareas en GitHub Projects (Todo / In Progress / Done) con responsable, cerradas desde los commits con closes #N. Evolución: T04 y T06 reasignadas, issue duplicada cerrada y T37 añadida por el cambio. Capturas inicial y final en el Anexo.

Prácticas aplicadas: (1) **Kanban con límite de trabajo en curso** (2 por integrante), que permitió coordinar un trabajo asincrónico sin reuniones; (2) **pruebas automatizadas en cada incremento con commits vinculados a tareas**, que permitió recibir el cambio sobre una versión estable. **Descartada: programación en pareja**, imposible en un parcial asincrónico con horarios distintos; se reemplazó por revisión cruzada asíncrona, que encontró un defecto real.

Definición de terminado: resultado en main; si es código, pytest completo en verde; vocabulario común (escena, opción, condición, efecto, estado, sesión); tarjeta en Done; uso de IA registrado en IA.md.

Aprendizaje y mejora: perdimos cerca de una hora intentando crear el tablero con scripts que no funcionaban en Windows, y varios commits hechos desde la web quedaron sin ID de tarea. **Acción:** verificar herramientas en el sistema real de ambos antes de automatizar y acordar desde el primer commit la convención Txx: descripción (closes #N).

3. Requerimientos

Vocabulario común: **historia** (archivo), **escena**, **opción**, **condición** (igualdad booleana que habilita), **efecto** (asignación booleana), **estado** (variables actuales), **sesión** (partida). Una opción está **habilitada** si cumple todas sus condiciones.

ID	N	Requisito	Criterio de aceptación
RF01	N01	Cargar una historia desde la ruta de un JSON	La historia de referencia carga con su título y 6 escenas
RF02	N01	Iniciar en inicio con las variables del archivo	Sesión nueva: E1, ambas variables en falso
RF03	N01	Ejecutar cualquier historia válida sin modificar el código	La segunda historia se juega con el mismo comando, sin cambios en src/
RF04	N02	Mostrar el texto de la escena actual	Sesión nueva muestra el texto de E1
RF05	N02	Mostrar solo las opciones habilitadas	**CA01**: tras revisar e investigar, E3 muestra solo Activar con código y Cortar
RF06	N03	Aplicar los efectos de la opción elegida	En E2, Investigar deja conoce_origen = verdadero
RF07	N03	Avanzar al destino de la opción elegida	En E2, Investigar deja la escena en E3
RF08	N03	El motor rechaza opción inexistente o no habilitada sin cambios	**CA02**: enviar activar_codigo sin conocer el origen se rechaza; escena y estado iguales
RF09	N03	La interfaz rechaza entradas que no son una opción mostrada	abc, vacío o 9 muestran aviso y repiten las opciones
RF10	N04	Conservar el estado durante toda la sesión	conoce_origen de E2 habilita Activar con código en E3
RF11	N05	Reconocer el final y no ofrecer avance	Cortar comunicación muestra E6 con indicación de fin
RF12	N05	Reiniciar con escena y variables iniciales	Desde E4, reiniciar deja E1 y variables en falso
RF13	N05	Informar bloqueo y ofrecer reinicio	Escena no final sin opciones habilitadas muestra aviso de bloqueo

ID	N	Requisito	Criterio de aceptación
RF14	N06	Validar todo antes de abrir sesión	**CA03**: destino E99 rechaza la carga y no hay sesión
RF15	N06	Cada error indica tipo, causa y campo o ID si es identificable	CA03 menciona V06, E3, activar_codigo y E99
RF16	N06	Detectar las nueve categorías V01–V09	Un archivo inválido por categoría, rechazado con su código
RF17	N07	Deshacer la última elección válida (también desde final o bloqueo)	E2 -investigar- E3; deshacer: E2 con conoce_origen falso. Desde E4/E5/E6 vuelve a E3
RF18	N07	Un solo nivel de retroceso	Segundo deshacer se rechaza; una nueva elección lo vuelve a habilitar
RF19	N07	Sin retroceso al comenzar; reiniciar borra el historial	Sesión nueva o reiniciada: deshacer se rechaza
RF20	N07	Entrada inválida no altera ni reemplaza el punto de retroceso	Tras investigar, xyz se rechaza y deshacer vuelve a E2

Errores de contenido (RF16): V01 archivo ilegible o raíz no objeto · V02 campo obligatorio ausente o de tipo incorrecto · V03 inicio inexistente · V04 escena repetida · V05 opción repetida en la escena · V06 destino inexistente · V07 variable no declarada · V08 valor no booleano · V09 opciones incompatibles con *final*. Se reportan todos en una sola validación (P2).

RNF	Requisito medible	Procedimiento	Resultado
RNF01 Modificabilidad	Agregar una historia: 0 líneas cambiadas en src/ y se juega hasta un final	scripts/verificar_rnf.py revisa el commit de la segunda historia y corre la prueba I2	Cumple: el commit que agrega la segunda historia no modifica src/ y la prueba I2 pasa.
RNF02 Claridad de errores	9/9 archivos inválidos rechazados con código y causa; ID o campo cuando sea identificable (V01: nombre de archivo)	Prueba parametrizada sobre tests/datos_invalidos/	Cumple: 9/9 archivos rechazados con su código y el ID o campo esperado (10 pruebas aprobadas).
RNF03 Confiabilidad	1000 entradas inválidas aleatorias (semilla 2026): 0 excepciones y estado idéntico en el 100 %	Prueba test_RNF03 con random.Random(2026)	Cumple: 1000/1000 entradas rechazadas, 0 excepciones, estado idéntico.

#	Pregunta al cliente	Respuesta o supuesto
P1	Conflicto: la escritora quiere retroceder siempre y el diseñador decisiones irreversibles. ¿Siempre, en escenas marcadas o nunca?	Acuerdo provisional en v1: sin retroceso. **Respondida en la hora 6**: deshacer solo la última decisión, un nivel (RF17–RF20)
P2	Si hay varios errores, ¿mostrar el primero o todos?	Supuesto: todos; se detiene solo si el archivo es ilegible o le falta la estructura base
P3	¿Una escena inalcanzable es un error?	Supuesto: no; la guía no exige analizar caminos. Limitación conocida

Revisión cruzada y defecto corregido (Javier): RNF02 exigía el ID en el 100 % de los casos, pero RF15 lo pide solo cuando es identificable y un archivo ilegible (V01) no tiene ID. Habría hecho fallar un validador correcto. Corrección: RNF02 exige el ID cuando sea identificable y, en V01, el nombre del archivo (docs/revision_cruzada.md).

4. Modelado

Modelo (Anexo)	Qué muestra	RF
Contexto	Motor como caja negra con escritor, lector y sistema de archivos	N01–N07
Casos de uso	Nueve casos; iniciar incluye cargar, que incluye validar; final y bloqueo extienden elegir; deshacer	RF01–RF20
Clases	Historia, Escena, Opción (con esta_habilitada), Sesión (anterior, deshacer), ResultadoEleccion, ErrorContenido	RF02–RF20
Secuencia: elegir opción	Filtro de la interfaz (RF09), verificación del motor (RF08/CA02), guardado de anterior, efectos y destino	RF05–RF09, RF11, RF13, RF17, RF20
Estados de la sesión	EnCurso, Finalizada, Bloqueada; elegir, reiniciar y deshacer	RF02, RF08, RF11–RF13, RF17–RF19

Decisiones: la validación ocurre antes de que exista la sesión; un rechazo no es una excepción sino un ResultadoEleccion; la regla de habilitación vive en Opcion, de modo que ningún componente conoce IDs concretos como E3. Los nombres de los modelos son los mismos del código.

5. Arquitectura

Capa	Componente	Responsabilidad	Depende de
Interfaz	consola.py, __main__.py	Mostrar escena y opciones, filtrar entrada (RF09), comandos jugar / validar, r, d, s	Carga, Motor
Motor	sesion.py, modelo.py	Opciones habilitadas, elegir, efectos, final, bloqueo, reinicio, deshacer	Nada externo
Carga y validación	cargador.py, validador.py	Leer el archivo, detectar V01–V09, construir la Historia	Modelo
Contenido	data/*.json	Las historias, editadas por los escritores	—

ADR-01. Capas con el contenido como datos externos. *Contexto:* la escritora debe cambiar la historia sin programar, la guía prohíbe código específico para E1 o E2 y se esperaba un cambio en la hora 6. *Alternativas:* historia escrita en el código (rápida, pero viola la guía), un único módulo (no permite probar el motor sin la consola) y capas con dependencias en un solo sentido. *Elección:* capas. *Consecuencias:* una historia nueva no toca src/ (RNF01), el motor se prueba aislado y el cambio quedó localizado; a cambio, más archivos y nombres que mantener coherentes.

ADR-02. Validación completa antes de crear la sesión. *Contexto:* confiabilidad y N06. *Alternativas:* validar perezosamente al llegar a cada escena (el error aparece en plena partida), detenerse en el primer error (el escritor corrige uno por vez) y validar todo al cargar. *Elección:* validar todo. *Consecuencias:* una historia inválida nunca llega al lector (CA03), el motor asume datos válidos y es más simple, el escritor ve todos sus errores; el costo de leer todo al inicio es despreciable con historias pequeñas.

Separación contenido / reglas / interfaz: el contenido solo vive en data/, las reglas solo en modelo.py y sesion.py, y la presentación solo en consola.py. Se demuestra con la segunda historia (La bóveda de la estación, 5 escenas), agregada en un commit que no toca src/, y con el cambio de la hora 6, que no tocó modelo, cargador, validador ni datos.

6. Implementación y pruebas

Python 3.10+ (probado en 3.14, Windows), interfaz de consola, ejecución local: python main.py jugar|validar <archivo>. **Bibliotecas:** solo biblioteca estándar (json, pathlib, dataclasses, argparse, random; licencia PSF) y **pytest** (MIT) para pruebas.

Decisión de diseño detallado (responsabilidades y acoplamiento): Sesion.elegir() devuelve un ResultadoEleccion en lugar de lanzar excepciones. El motor decide *si* se acepta y la consola decide *cómo* mostrarlo, así que la consola no conoce reglas y el motor no conoce la pantalla. Una entrada mala nunca cierra el programa (RNF03). La interfaz recibe funciones de entrada y salida, lo que permite probarla sin teclado.

Caso	Prueba (tests/)	Variantes
U1 Condiciones habilitan solo lo correcto	test_motor.py::test_U1_*	con y sin conoce_origen; condiciones vacías
U2 Elección válida: efectos y destino	test_motor.py::test_U2_*	—
U3 Opción inexistente o no habilitada deja intacto el estado	test_motor.py::test_U3_*	6 entradas, incluida CA02; elección en un final
U4 Errores de contenido	test_validador.py::test_U4_*	14 mutaciones V02–V09 + 5 de V01; CA03
U5 Reiniciar (extendida: borra historial)	test_motor.py::test_U5_*	—
U6 Deshacer (cambio)	test_motor.py::test_U6_*	restaura y consume; 4 entradas inválidas; nueva elección; inicio; desde E4/E5/E6; bloqueo
I1 Archivo real + motor + consola (extendida: deshacer desde final)	test_integracion.py::test_I1_*	motor directo; consola con entradas inválidas; consola con d

Resultado de la última ejecución: 68 de 68 aprobadas (U1–U6, I1, I2, RNF02, RNF03, CA01–CA03) (evidencias/pruebas/). Además se cubren los tres finales, el bloqueo, los comandos validar y la segunda historia (I2).

7. Gestión del cambio (hora 6)

Solicitud: el lector debe poder deshacer su última decisión, revirtiendo todas sus consecuencias; reemplaza el acuerdo provisional P1. **Versión de partida:** v1.0-pre-cambio (45 pruebas aprobadas). Nuevos requisitos RF17–RF20 (sección 3).

Artefacto afectado	Cambio
Requisitos y alcance	N07 con RF17–RF20; P1 respondida; retroceso de un nivel pasa a estar incluido
Modelos	Casos de uso (nuevo caso), clases (anterior, deshacer, puede_deshacer), secuencia (guardar anterior), estados (transiciones deshacer), componentes
Código	sesion.py: deshacer(), puede_deshacer(); elegir() y reiniciar() gestionan anterior. consola.py: comando d
Sin cambios	modelo.py, cargador.py, validador.py e historias JSON
Pruebas	U6 nueva con variantes; U5 e I1 extendidas
Backlog y trazabilidad	T37 nueva; T26–T28; matriz con RF17–RF20

Decisión: guardar **una sola copia** de escena y estado antes de cada elección válida, en lugar de una pila o de re-jugar la partida. El cliente pidió un solo nivel, así que la regla se cumple por construcción (al deshacer, la copia se vacía). Si mañana piden varios niveles, el cambio queda dentro de Sesion. **Impacto:** el cambio quedó contenido en las capas de motor e interfaz, como anticipaba ADR-01.

8. Trazabilidad

La matriz completa (RF01–RF20 y RNF01–RNF03 hacia necesidad, modelo, archivo/función y nombre exacto de la prueba) está en el Anexo y en docs/07_trazabilidad.md. Ejemplo: RF08 → Secuencia (alternativas de rechazo), Estados (autotransición EnCurso) → sesion.py · elegir() → test_U3_*[no_habilitada_CA02].

9. Aportes por integrante

Integrante	Tareas y archivos (ver historial de Git)
Sebastián Felipe Caicedo Acosta	T01 repositorio y estructura; T03–T10 fundamentos, proceso y requisitos (docs/01, docs/02); T14–T18 clases, secuencia, estados, componentes y ADR (docs/03, docs/04, diagramas 03–06); T19–T22 código y pruebas (src/, tests/); T25–T28 tag, análisis e implementación del cambio, trazabilidad (docs/05, docs/07); T29 verificación de RNF; T31 README; T32 informe y anexo; T33 video; T34 captura final y git log; T35 ZIP; T36 agilidad (docs/06); integración del JSON oficial
Javier Alejandro Carta Lacharme	T02 captura inicial del tablero; T11 revisión cruzada y defecto de RNF02 (docs/revision_cruzada.md); T12–T13 diagramas de contexto y casos de uso (diagramas 01–02, actualizados por el cambio); T23 archivos inválidos V01–V09 (tests/datos_invalidos/); T24 segunda historia (data/segunda_historia.json)

10. Fuentes y herramientas de apoyo

Claude (Anthropic), usado por Sebastián, para borradores de documentación, diagramas PlantUML, código y pruebas. **ChatGPT (OpenAI)**, usado por Javier, para la revisión cruzada y los diagramas de contexto y casos de uso. Todo resultado se verificó ejecutando las pruebas, contrastando con la guía y la rúbrica, y revisando la coherencia entre modelos y código. El registro detallado por punto (qué se pidió, qué se usó, cómo se verificó) está en IA.md. Otras herramientas: Python, pytest, Git/GitHub, GitHub Projects y PlantUML. No se reutilizaron fragmentos de código de terceros.